

# 4-bit ALU with shift-based multiply and divide

This report covers the theory of arithmetic logic units, the design of a 4-bit ALU with ten operations, and how multiplication and division are done by shifting and re-using the ALU's own adder instead of adding a separate multiplier or divider circuit. It explains every source file, how the files connect, and how the design was verified.

|              |                                                                                   |
|--------------|-----------------------------------------------------------------------------------|
| Operations   | ADD, SUB, AND, OR, XOR, NOT, SHL, SHR (single cycle) · MUL, DIV (multi-cycle)     |
| Data width   | 4-bit inputs A and B · 8-bit result (holds a full product)                        |
| Flags        | carry, zero, overflow, negative, div_by_zero · status: busy, done                 |
| Source files | 9 Verilog design files + 1 testbench, ready to add to a Vivado project            |
| Verification | All 4,096 input cases checked by the testbench, 0 errors; lint-clean in Verilator |
| Size check   | Synthesised with Yosys 0.33 to count gates and flip-flops                         |

|   |                                        |    |                             |
|---|----------------------------------------|----|-----------------------------|
| 1 | What an ALU is                         | 8  | The complete ALU            |
| 2 | Binary numbers inside an ALU           | 9  | The files and how they link |
| 3 | Building blocks                        | 10 | Gate-level diagrams         |
| 4 | Multiplication by shifting             | 11 | Verification                |
| 5 | Division by shifting                   | 12 | How to run it               |
| 6 | Why shifting makes the circuit smaller | A  | Appendix: full source code  |
| 7 | Control and timing                     |    |                             |

## 1 What an ALU is

The **Arithmetic Logic Unit (ALU)** is the part of a processor that does the actual calculating. Every instruction that adds, subtracts, compares, masks bits or shifts a value is carried out by the ALU. The rest of the processor (the control unit, registers and memory) exists to feed the ALU with operands and to store what it produces.

An ALU has three kinds of connections:

- **Operands** — the data to work on. Here: two 4-bit numbers, **a** and **b**.
- **Opcode** — a small binary code that says which operation to do. Here: 4 bits, so up to 16 operations (10 are used).
- **Outputs** — the **result** and a set of **status flags** that describe it (was there a carry? is it zero? is it negative? did it overflow?). A processor uses flags to make decisions, for example "branch if zero".

Inside, an ALU is split into a **datapath** (the circuits that compute: adder, logic gates, shifter) and **control** (the circuits that decode the opcode and choose which datapath output becomes the result). A key idea in ALU design is that *every* datapath unit works at the same time on the same inputs; a multiplexer then picks the one answer that was asked for. This is simpler and faster than switching units on and off.

### Combinational and sequential operations

A **combinational** circuit has no memory: its output depends only on its current inputs and appears after a short gate delay. ADD, SUB, AND, OR, XOR, NOT and the shifts are combinational — the answer is ready in the same clock cycle. A **sequential** circuit has flip-flops that store state and change only on a clock edge. This design does MUL and DIV sequentially: one small step per clock, four steps in total. That is the price paid for a much smaller circuit (Section 6).

## 2 Binary numbers inside an ALU

### Unsigned and two's complement

Four bits can hold 16 patterns. Read as **unsigned** numbers they mean 0 to 15. Read as **two's complement** (signed) numbers the top bit has weight  $-8$ , so the range is  $-8$  to  $+7$ . The same adder circuit works for both — only the interpretation of the result and the flags differ. That is why an ALU provides both a **carry** flag (for unsigned results) and an **overflow** flag (for signed results).

| BITS | UNSIGNED | SIGNED (TWO'S COMPLEMENT) | BITS | UNSIGNED | SIGNED |
|------|----------|---------------------------|------|----------|--------|
| 0000 | 0        | 0                         | 1000 | 8        | -8     |
| 0011 | 3        | +3                        | 1011 | 11       | -5     |
| 0111 | 7        | +7                        | 1111 | 15       | -1     |

## Subtraction is addition

To negate a two's complement number, invert every bit and add 1:  $-B = \sim B + 1$ . So

$$A - B = A + (\sim B) + 1$$

This is the reason an ALU does not need a separate subtractor. The adder gets an extra control input, `sub`. Each bit of B passes through an XOR gate with `sub`: when `sub = 0` the bit passes unchanged, when `sub = 1` it is inverted (because  $x \oplus 1 = \sim x$ ). The "+1" is supplied by feeding `sub` into the adder's carry-in. One adder therefore does both ADD and SUB.

## Carry versus overflow

- **Carry** is the fifth bit that falls out of a 4-bit addition. For unsigned numbers it means the true answer was bigger than 15. For subtraction, carry = 1 means *no borrow* was needed ( $A \geq B$ ).
- **Overflow** means the signed answer does not fit in  $-8 \dots +7$ . It happens when two numbers of the same sign are added and the result has the other sign ( $5 + 3 = 8$  appears as `1000` = -8). In hardware it is simply `c3  $\oplus$  c4`: the carry into the top bit differs from the carry out of it.

## 3 Building blocks

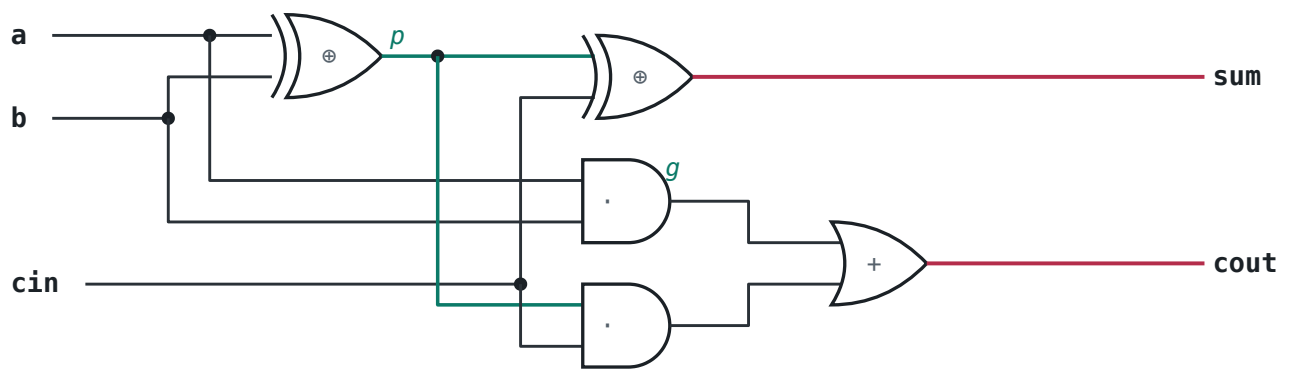
### 3.1 Full adder

A full adder adds three single bits (a, b and a carry-in) and gives a sum bit and a carry-out. From the truth table:

| A | B | CIN | SUM | COUT |
|---|---|-----|-----|------|
| 0 | 0 | 0   | 0   | 0    |
| 0 | 0 | 1   | 1   | 0    |
| 0 | 1 | 0   | 1   | 0    |
| 0 | 1 | 1   | 0   | 1    |
| 1 | 0 | 0   | 1   | 0    |
| 1 | 0 | 1   | 0   | 1    |
| 1 | 1 | 0   | 0   | 1    |
| 1 | 1 | 1   | 1   | 1    |

```
p    = a  $\oplus$  b          (propagate)
g    = a  $\cdot$  b          (generate)
sum  = p  $\oplus$  cin
cout = g + p  $\cdot$  cin
```

The sum is 1 when an odd number of inputs are 1. A carry is *generated* when a and b are both 1, and an incoming carry is *propagated* when exactly one of them is 1. The circuit needs 2 XOR, 2 AND and 1 OR gate (`full_adder.v`).



**Figure 1.** Full adder. Five gates: two XOR for the sum, two AND and one OR for the carry.

### 3.2 Ripple-carry adder / subtractor

Four full adders are chained: the carry-out of bit  $i$  is the carry-in of bit  $i+1$ . This is a **ripple-carry adder**. It is the smallest possible adder, but the carry must travel ("ripple") through all four stages, so the delay grows with the width. The XOR on each B input and `sub` as C0 turn it into an adder/subtractor (`adder_subtractor_4bit.v`). It also brings out C3 and C4 for the overflow flag.

### 3.3 Logic unit

Bitwise operations act on each bit position independently: bit  $i$  of the result depends only on bit  $i$  of A and B. AND is used to clear bits (masking), OR to set bits, XOR to flip bits or compare, NOT to invert. Each needs one gate per bit (`logic_unit_4bit.v`).

### 3.4 Shifter

A logical shift slides every bit one place. Shifting left moves each bit to the next higher position and puts a 0 in bit 0; the old top bit falls out. Shifting right does the opposite. Because each binary position is worth twice the one below it:

|                                 |                                 |                |
|---------------------------------|---------------------------------|----------------|
| shift left by 1 = multiply by 2 | 1011 (11) → 0110 (6), bit out 1 | (22 = 16 + 6)  |
| shift right by 1 = divide by 2  | 1011 (11) → 0101 (5), bit out 1 | (11 = 2·5 + 1) |

A shift needs **no gates at all** — it is only wiring: output bit  $i$  is connected to input bit  $i-1$  (left) or  $i+1$  (right). In Verilog this is written with concatenation: `{a[2:0], 1'b0}` and `{1'b0, a[3:1]}` (`shifter_4bit.v`). The bit that falls out is sent to the carry flag. Shifting is what makes the small multiplier and divider in Sections 4 and 5 possible.

### 3.5 Decoder and multiplexer

A **decoder** turns a binary code into separate "select" lines; each is an AND gate that matches one pattern of the opcode bits (`control_unit.v`). A **multiplexer** (mux) has several data inputs and passes one of them to the output according to a select code. The result multiplexer (`result_mux.v`) chooses between the adder, logic, shifter and multiply/divide outputs. At gate

level a mux is an AND-OR network: each input is ANDed with its decoded select line and the products are ORed together.

### 3.6 Status flags

| FLAG        | MEANING                                                        | LOGIC                                   |
|-------------|----------------------------------------------------------------|-----------------------------------------|
| carry       | ADD: carry out · SUB: 1 = no borrow · SHL/SHR: bit shifted out | <code>arith·C4 + shl·a3 + shr·a0</code> |
| overflow    | signed result does not fit (ADD/SUB only)                      | <code>arith·(C3 ⊕ C4)</code>            |
| zero        | all 8 result bits are 0                                        | <code>NOR(result[7:0])</code>           |
| negative    | sign bit of a 4-bit result (0 for MUL/DIV, which are unsigned) | <code>¬muldiv · result[3]</code>        |
| div_by_zero | DIV with b = 0                                                 | stored when DIV starts                  |

## 4 Multiplication by shifting

### The idea: long multiplication in binary

Multiplying on paper means: for each digit of the multiplier, write down the multiplicand times that digit, shifted left by the digit's position, then add all the rows. In binary every digit is 0 or 1, so each row is either **0** or **the multiplicand itself, shifted**. Multiplication therefore needs only two things: shifting, and adding.

```

      1101      a = 13 (multiplicand)
    × 1011      b = 11 (multiplier)
    -----
      1101      b[0] = 1 → add 1101
      1101      b[1] = 1 → add 1101 shifted left 1
      0000      b[2] = 0 → add nothing
    + 1101      b[3] = 1 → add 1101 shifted left 3
    -----
    10001111    = 143
```

A combinational **array multiplier** builds this whole picture at once: 16 AND gates make the 16 partial product bits and a grid of adders sums the rows. The size grows with the *square* of the width ( $n^2$  AND gates, about  $n^2$  adder cells).

### The sequential shift-and-add algorithm

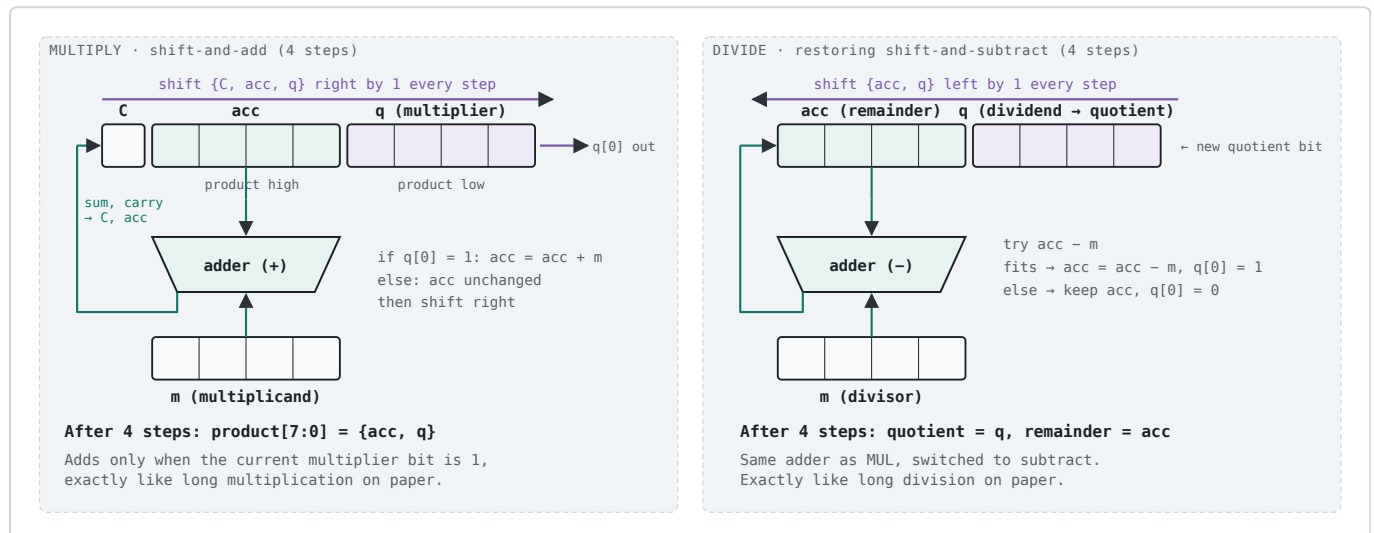
Instead of adding all rows at once, the rows can be added **one per clock cycle** using a single adder. Rather than shifting the multiplicand left, the hardware keeps the multiplicand still and shifts the running total *right*, which is equivalent and keeps the adder only 4 bits wide. Three 4-bit registers are used: `acc` (running total, becomes the high half of the product), `q` (holds the multiplier; its empty bits fill up with the low half of the product) and `m` (the multiplicand), plus one carry bit.

```

load:  acc = 0,  q = b (multiplier),  m = a (multiplicand)
repeat 4 times:
    if q[0] == 1:  {C, acc} = acc + m      (add the multiplicand)
    else:         C = 0                  (add nothing)
    {C, acc, q} = {C, acc, q} >> 1      (shift everything right)
product = {acc, q}

```

Each step looks at the lowest bit of `q` (the current multiplier bit), adds if it is 1, and then shifts. The shift does two jobs at once: it moves the next multiplier bit into `q[0]`, and it moves the finished low bit of the product into the top of `q`.



**Figure 2.** Multiply and divide datapaths. Both use the same three registers and the same adder. Multiply shifts right and adds; divide shifts left and subtracts.

## Worked example: 13 × 11

These register values were recorded from the simulation of `muldiv_unit.v`, one row per clock:

| CLOCK              | WHAT HAPPENS                                         | ACC       | Q       |
|--------------------|------------------------------------------------------|-----------|---------|
| load               | acc = 0000, q = b = 1011, m = a = 1101               | 0000      | 1011    |
| step 1             | q[0]=1 → acc + m = 0000 + 1101 = 0 1101, shift right | 0110      | 1101    |
| step 2             | q[0]=1 → acc + m = 0110 + 1101 = 1 0011, shift right | 1001      | 1110    |
| step 3             | q[0]=0 → no add, just shift right                    | 0100      | 1111    |
| step 4             | q[0]=1 → acc + m = 0100 + 1101 = 1 0001, shift right | 1000      | 1111    |
| product = {acc, q} |                                                      | 1000 1111 | = 143 ✓ |

## 5 Division by shifting

### The idea: long division in binary

Long division brings down one digit of the dividend at a time and asks "does the divisor fit into what I have so far?" In binary the answer is only yes (quotient bit 1, subtract the divisor) or no

(quotient bit 0, subtract nothing). So division needs shifting, subtracting and a comparison — and the comparison comes for free: the subtraction's borrow tells whether the divisor fit.

### The restoring division algorithm

```
load:  acc = 0 (remainder),  q = a (dividend),  m = b (divisor)
repeat 4 times:
    {acc, q} = {acc, q} << 1                (bring down the next dividend bit)
    try  acc - m
    if it fits (no borrow):  acc = acc - m,  q[0] = 1
    else:                    acc unchanged,  q[0] = 0    ("restore")
quotient = q,  remainder = acc
```

As the dividend bits are shifted out of the top of `q`, the quotient bits are shifted into its bottom, so the same register holds both. The name *restoring* comes from the textbook version that subtracts first and adds the divisor back when the result is negative; this design simply does not write the difference back, which has the same effect and needs no second addition.

One detail: after the shift the partial remainder can need 5 bits (up to  $2 \times 14 + 1 = 29$ ). The bit shifted out of the top of `acc` is kept as `r_top`. If it is 1 the value is at least 16, which is larger than any 4-bit divisor, so the divisor certainly fits. The "fits" test is therefore `fits = r_top | C4`, where `C4 = 1` means the subtraction needed no borrow.

### Worked example: 13 ÷ 4

| CLOCK                         | WHAT HAPPENS                                     | ACC                                           | Q    |
|-------------------------------|--------------------------------------------------|-----------------------------------------------|------|
| load                          | acc = 0000, q = a = 1101, m = b = 0100           | 0000                                          | 1101 |
| step 1                        | shift → acc = 0001; 1 - 4 does not fit → q bit 0 | 0001                                          | 1010 |
| step 2                        | shift → acc = 0011; 3 - 4 does not fit → q bit 0 | 0011                                          | 0100 |
| step 3                        | shift → acc = 0110; 6 - 4 = 2 fits → q bit 1     | 0010                                          | 1001 |
| step 4                        | shift → acc = 0101; 5 - 4 = 1 fits → q bit 1     | 0001                                          | 0011 |
| quotient = q, remainder = acc |                                                  | q = 0011 = 3, acc = 0001 = 1 ✓ (13 = 4·3 + 1) |      |

**Divide by zero.** With `m = 0` the divisor always "fits", so the algorithm naturally returns quotient 1111 and remainder = `a`. The unit also sets `div_by_zero = 1` so the user can detect it.

## 6 Why shifting makes the circuit smaller

Two decisions keep the multiplier and divider small:

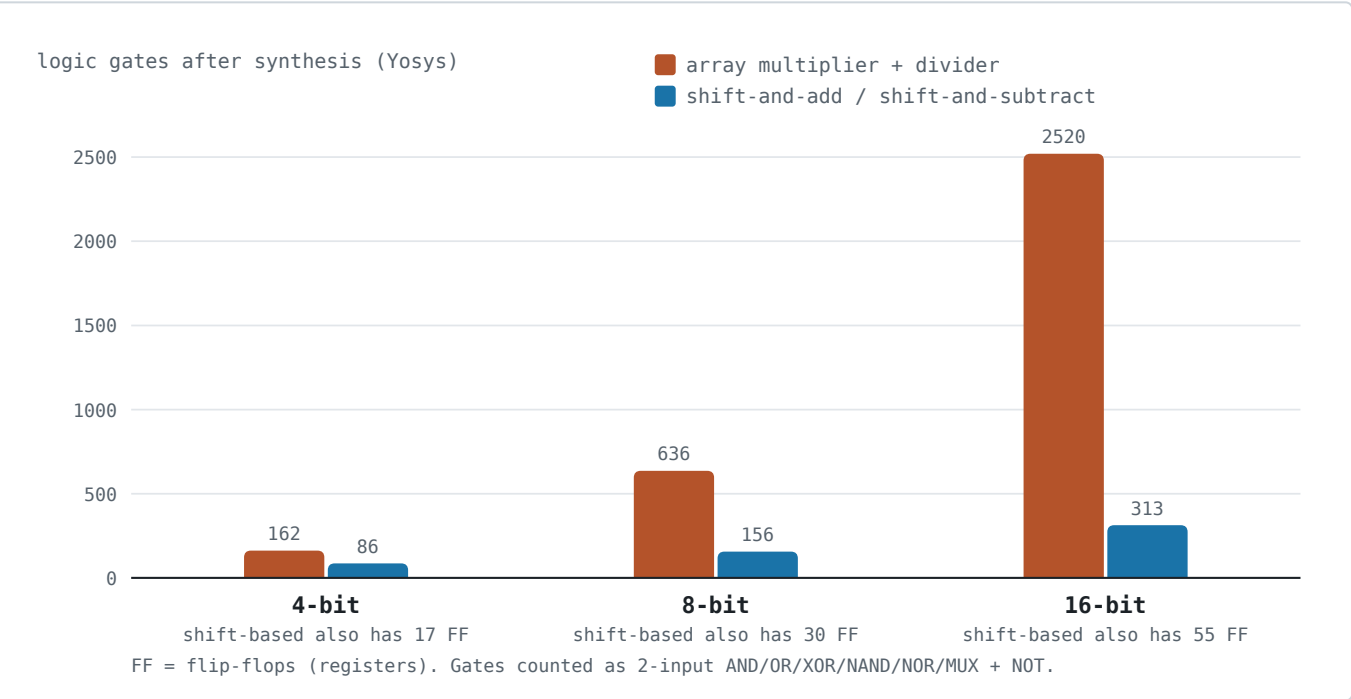
1. **Time instead of space.** An array multiplier or divider does all `n` steps at once, so it needs about  $n^2$  cells. The sequential version does one step per clock with one `n`-bit adder, so its size grows only linearly with `n`.

2. **Hardware sharing.** MUL needs an adder and DIV needs a subtractor — the ALU already has exactly that. So `muldiv_unit` contains *no adder of its own*. While it is busy, two small multiplexers in `alu_4bit.v` connect its registers to the ALU's adder/subtractor; the rest of the time the adder serves ADD and SUB as normal. MUL and DIV also share the same three registers and the same counter.

To check the claim with real numbers, the designs were synthesised with the open-source tool Yosys and mapped to simple 2-input gates. Results:

| DESIGN                                           | LOGIC GATES | FLIP-FLOPS |
|--------------------------------------------------|-------------|------------|
| Original 8-operation ALU (no MUL/DIV)            | 125         | 0          |
| ALU + combinational array multiplier and divider | 289         | 0          |
| <b>ALU + shift-based MUL/DIV (this design)</b>   | <b>193</b>  | <b>18</b>  |

Adding MUL and DIV as arrays costs 164 extra gates. Adding them by shifting costs 68 extra gates and 18 flip-flops. A flip-flop is larger than a simple gate — roughly the area of 4 to 6 gates in a typical standard-cell library — so **at 4 bits the two approaches end up about the same total area**. The real benefit appears as the width grows, because the array grows with  $n^2$  and the shift-based unit grows with  $n$ :

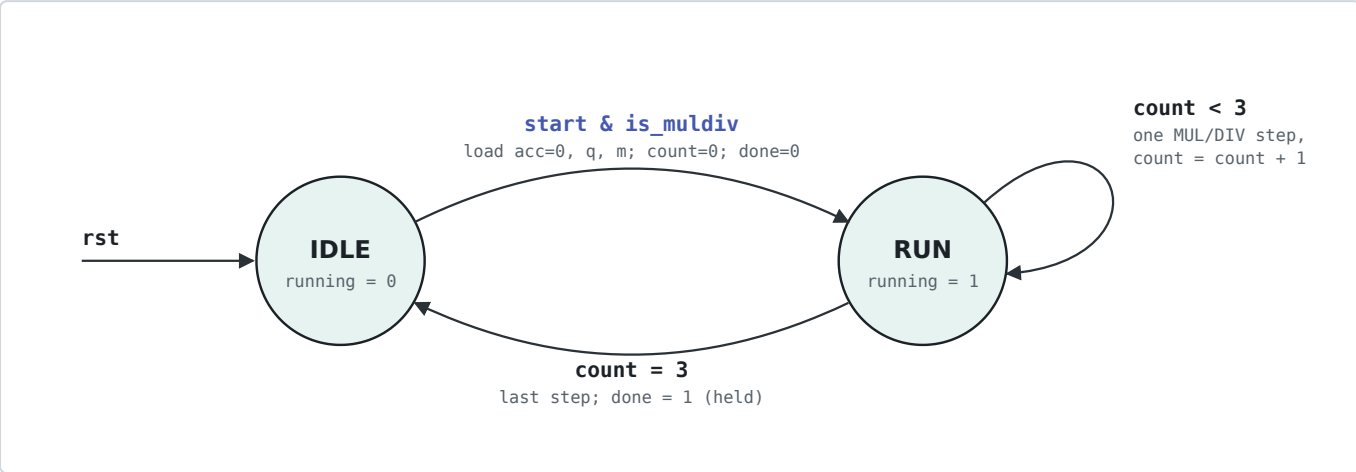


**Figure 3.** Multiplier + divider size versus data width. Standalone units synthesised with Yosys (the shift-based unit here includes its own adder). At 8 bits the shift-based unit is about 2× smaller and at 16 bits about 4× smaller, counting each flip-flop as 5 gates.

**The trade-off is speed.** The array gives its answer after one (long) combinational delay. The shift-based unit needs 1 clock to load plus 4 clocks to compute. This is the classic area-versus-time trade-off, and it is how many small processors and microcontrollers implement multiply and divide.

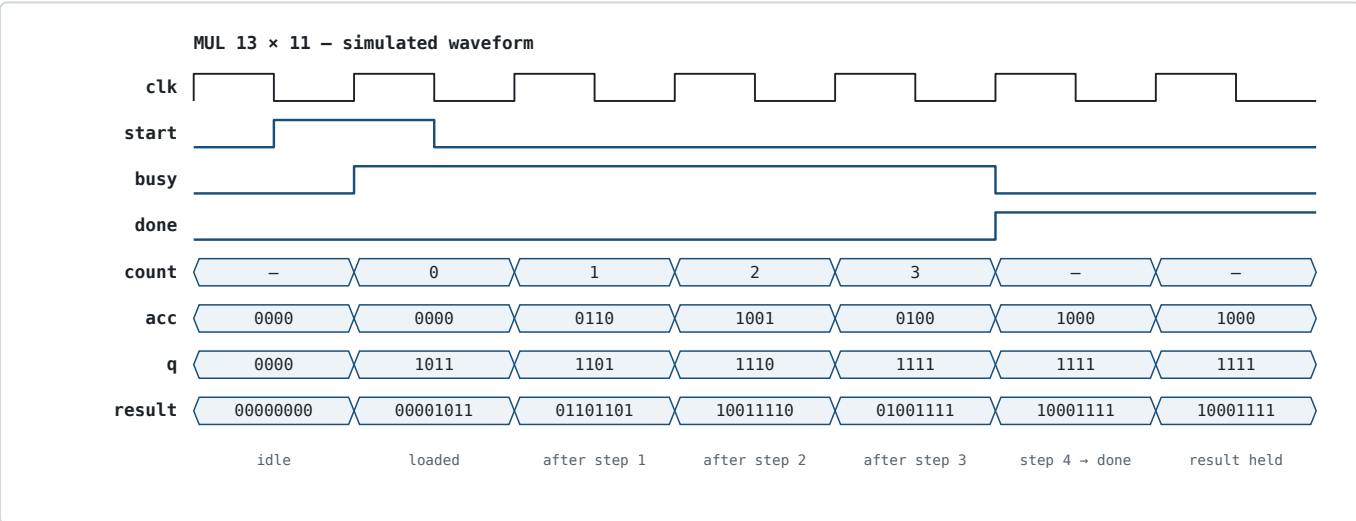
# 7 Control and timing

`muldiv_unit` is controlled by a two-state machine and a 2-bit counter. In **IDLE** the unit waits. When `start` is 1 and the opcode is MUL or DIV, it loads the registers and goes to **RUN**. Each clock in RUN performs one step and increments the counter; after the fourth step it returns to IDLE and sets `done`. The result stays in the registers, and on the output, until the next start.

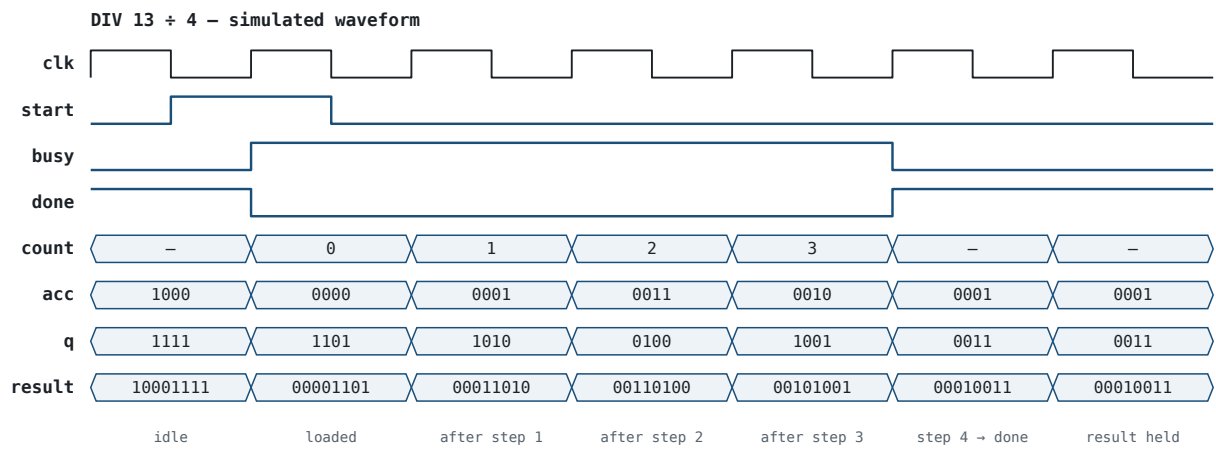


**Figure 4.** Control state machine. Two states plus a step counter. busy = 1 in RUN; done is set by the last step and cleared by the next start.

**How to use it:** set `a`, `b` and `opcode`, hold `start` high for one clock edge, then wait until `done` = 1 and read `result`. Keep `opcode` set to MUL or DIV while reading, because the result multiplexer selects by opcode. Single-cycle operations do not use `start`; their result appears straight away.



**Figure 5.** Multiply timing. Values after each rising clock edge, from the simulation. The product 1000 1111 (143) is ready when `done` rises and is held afterwards.



**Figure 6.** Divide timing. result = {remainder, quotient} = 0001 0011, i.e. remainder 1 and quotient 3.

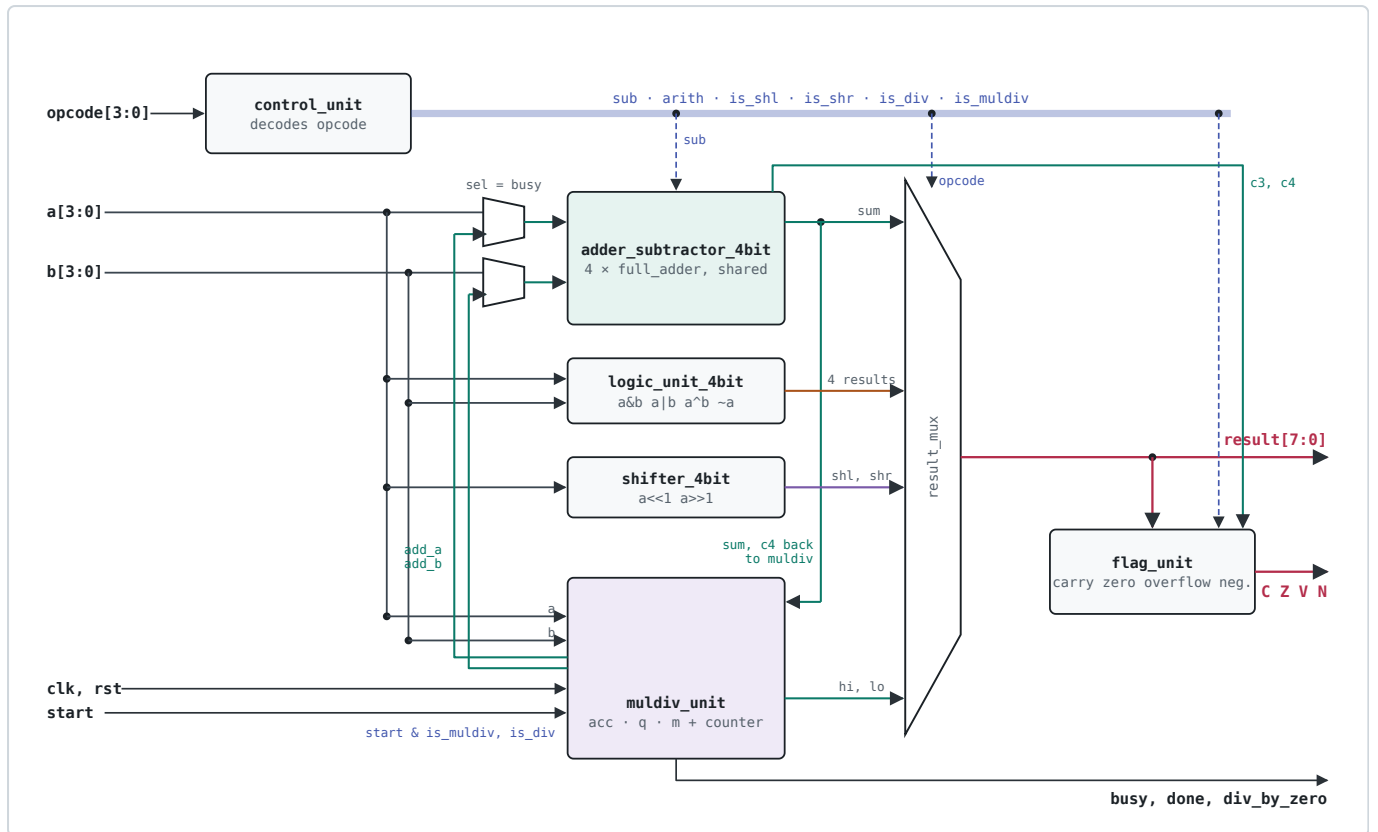
## 8 The complete ALU

### Opcodes

| OPCODE    | OPERATION | RESULT [7:0]          | CYCLES | FLAGS AFFECTED             |
|-----------|-----------|-----------------------|--------|----------------------------|
| 0000      | ADD       | {0000, a + b}         | 1      | C, V, Z, N                 |
| 0001      | SUB       | {0000, a - b}         | 1      | C (1 = no borrow), V, Z, N |
| 0010      | AND       | {0000, a & b}         | 1      | Z, N                       |
| 0011      | OR        | {0000, a   b}         | 1      | Z, N                       |
| 0100      | XOR       | {0000, a ^ b}         | 1      | Z, N                       |
| 0101      | NOT       | {0000, ~a}            | 1      | Z, N                       |
| 0110      | SHL       | {0000, a << 1}        | 1      | C (bit out), Z, N          |
| 0111      | SHR       | {0000, a >> 1}        | 1      | C (bit out), Z, N          |
| 1000      | MUL       | a × b (8 bits)        | 1 + 4  | Z                          |
| 1001      | DIV       | {remainder, quotient} | 1 + 4  | Z, div_by_zero             |
| 1010–1111 | unused    | 0000 0000             | –      | Z = 1                      |

### Ports of alu\_4bit

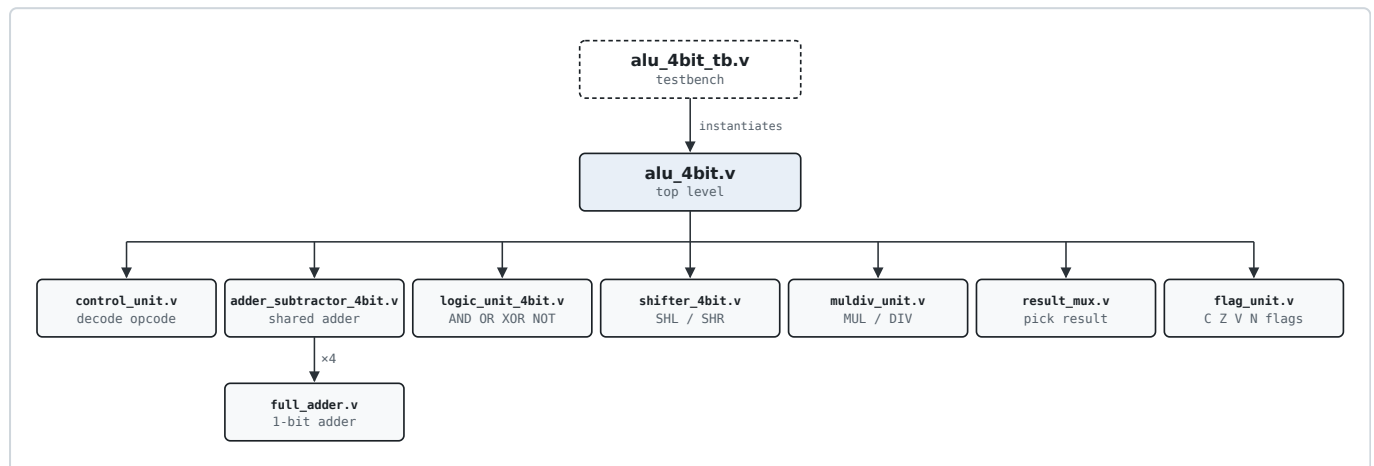
| PORT                            | DIR | WIDTH  | DESCRIPTION                          |
|---------------------------------|-----|--------|--------------------------------------|
| clk                             | in  | 1      | Clock (used only by MUL/DIV)         |
| rst                             | in  | 1      | Asynchronous reset, active high      |
| start                           | in  | 1      | Pulse for one clock to begin MUL/DIV |
| a, b                            | in  | 4 each | Operands                             |
| opcode                          | in  | 4      | Operation select (table above)       |
| result                          | out | 8      | Answer                               |
| carry, zero, overflow, negative | out | 1 each | Status flags                         |
| div_by_zero                     | out | 1      | Last DIV had b = 0                   |
| busy, done                      | out | 1 each | MUL/DIV running / finished           |



**Figure 7.** Top-level block diagram (alu\_4bit.v). All units see a and b at the same time; result\_mux picks one output. The highlighted adder is shared: the two muxes in front of it hand it to muldiv\_unit while busy = 1.

## 9 The files and how they link

The design is split into one module per file. `alu_4bit.v` is the top level: it contains no logic of its own except the two adder input multiplexers, and its job is to instantiate the other modules and wire them together. The testbench instantiates `alu_4bit`. In Vivado the nine design files are added as **Design Sources** and the testbench as a **Simulation Source**.



**Figure 8.** File hierarchy. An arrow means “instantiates”. `full_adder` is used four times inside `adder_subtractor_4bit`.

### What each file does

#### `alu_4bit.v` — top level

Declares the external ports, instantiates the seven sub-modules and connects them. It contains the only “glue” logic: `add_in_a = busy ? md_add_a : a` (and the same for `b` and `sub`), which hands the adder to the multiply/divide unit while it runs. It also gates `div_by_zero` with `is_div` and sends `start` to `muldiv_unit` only when the opcode is MUL or DIV.

#### `control_unit.v` — opcode decoder

Turns the 4-bit opcode into six one-bit control signals. `sub` goes to the adder, `arith/is_shl/is_shr/is_muldiv` go to `flag_unit`, and `is_div/is_muldiv` go to `muldiv_unit`. Pure combinational logic.

#### `adder_subtractor_4bit.v` and `full_adder.v` — the shared adder

`adder_subtractor_4bit` XORs `B` with `sub` and chains four `full_adder` instances. Its `sum` goes to `result_mux` (for ADD/SUB) and back to `muldiv_unit` (for MUL/DIV steps). `C4` goes to both `flag_unit` and `muldiv_unit`; `C3` goes to `flag_unit` for overflow.

#### `logic_unit_4bit.v`

Computes `a & b`, `a | b`, `a ^ b` and `~a` in parallel and sends all four to `result_mux`.

#### `shifter_4bit.v`

Produces `a << 1` and `a >> 1` (to `result_mux`) and the two bits shifted out (to `flag_unit` for the carry flag). Only wiring, no gates.

### **muldiv\_unit.v — sequential multiplier/divider**

Holds the `acc`, `q` and `m` registers, the step counter and the IDLE/RUN control. It drives the shared adder through `add_a`, `add_b`, `add_sub` and reads back `add_sum` and `add_c4`. Its `hi/lo` outputs go to `result_mux`; `busy` controls the adder input muxes in the top level; `busy`, `done` and `div_by_zero` also leave the ALU as outputs. The only clocked file in the design.

### **result\_mux.v**

Selects the 8-bit result by opcode from all the unit outputs. 4-bit answers are padded with four zeros.

### **flag\_unit.v**

Computes carry, overflow, zero and negative from the result, the adder carries, the shifted-out bits and the control signals.

### **alu\_4bit\_tb.v — testbench**

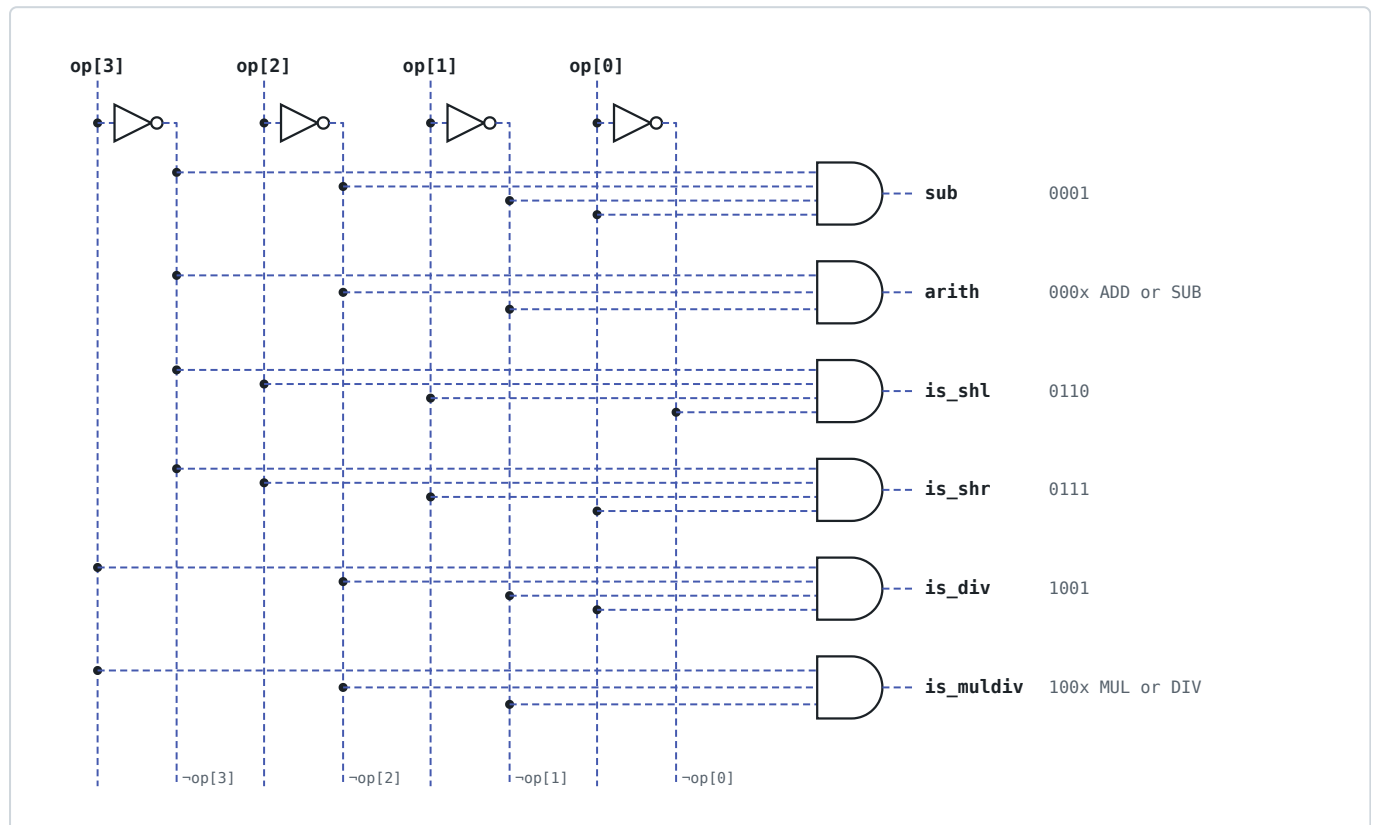
The testbench drives the top-level ports, prints examples and checks every input combination against a reference model (Section 11). It is only for simulation; it is never synthesised into hardware.

## Signal connections between files

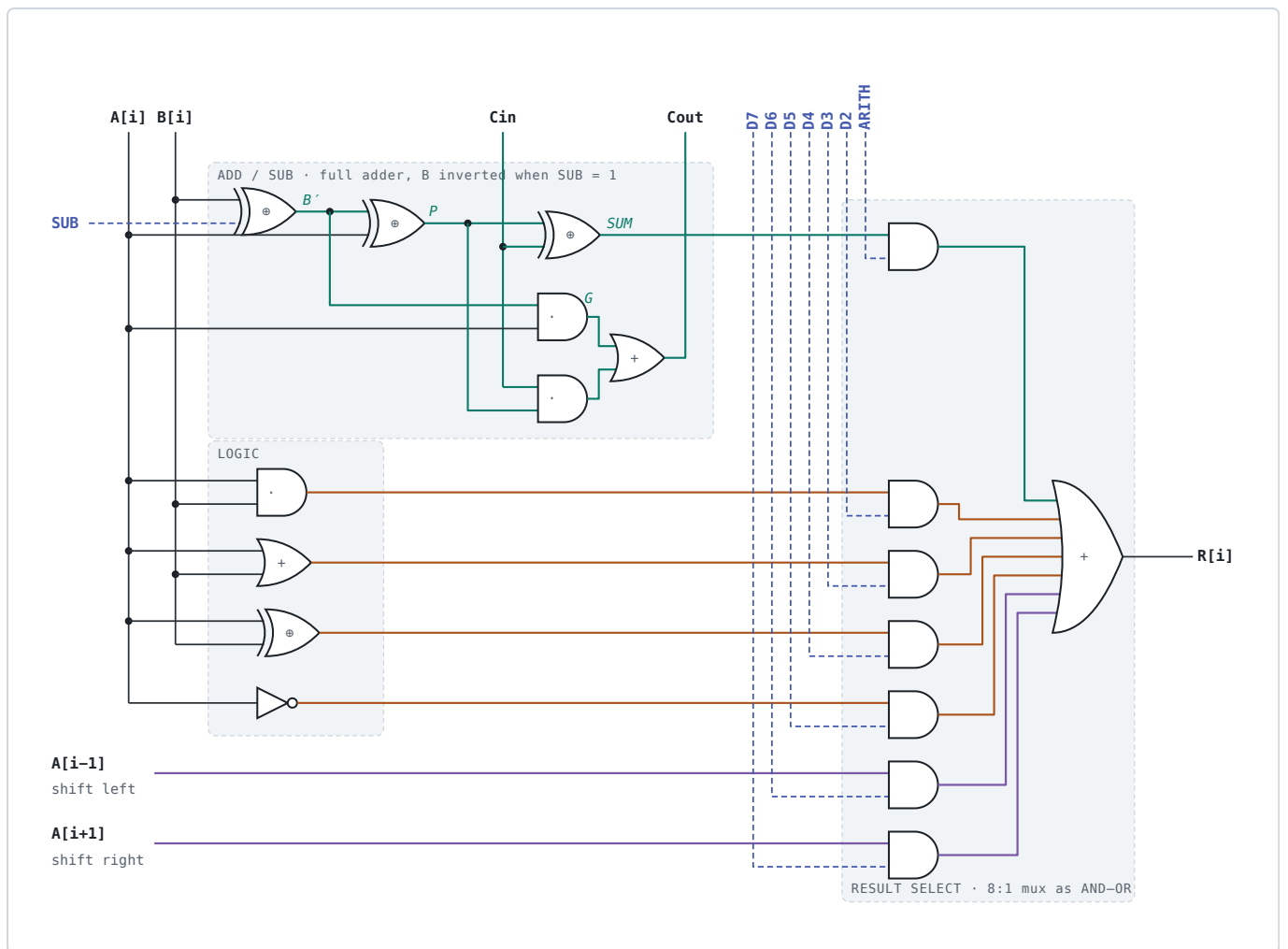
| SIGNAL                          | PRODUCED IN      | USED IN                                           | PURPOSE                       |
|---------------------------------|------------------|---------------------------------------------------|-------------------------------|
| opcode                          | port (testbench) | control_unit, result_mux                          | which operation               |
| a, b                            | port (testbench) | adder (via mux), logic_unit, shifter, muldiv_unit | operands                      |
| sub                             | control_unit     | alu_4bit mux → adder_subtractor                   | add or subtract               |
| arith, is_shl, is_shr           | control_unit     | flag_unit                                         | which flags apply             |
| is_div, is_muldiv               | control_unit     | muldiv_unit, flag_unit, alu_4bit                  | start / mode / gating         |
| add_a, add_b, add_sub           | muldiv_unit      | alu_4bit mux → adder_subtractor                   | borrow the adder              |
| busy                            | muldiv_unit      | alu_4bit (mux select), port                       | adder owner / status          |
| sum                             | adder_subtractor | result_mux, muldiv_unit                           | ADD/SUB result, MUL/DIV step  |
| c4                              | adder_subtractor | flag_unit, muldiv_unit                            | carry / borrow                |
| c3                              | adder_subtractor | flag_unit                                         | overflow detection            |
| and_out ... not_out             | logic_unit       | result_mux                                        | logic results                 |
| shl, shr                        | shifter          | result_mux                                        | shift results                 |
| shl_out_bit, shr_out_bit        | shifter          | flag_unit                                         | carry on shifts               |
| hi, lo                          | muldiv_unit      | result_mux                                        | product / remainder, quotient |
| result                          | result_mux       | flag_unit, port                                   | final answer                  |
| carry, zero, overflow, negative | flag_unit        | port                                              | status                        |
| done, div_by_zero               | muldiv_unit      | port                                              | status                        |

## 10 Gate-level diagrams

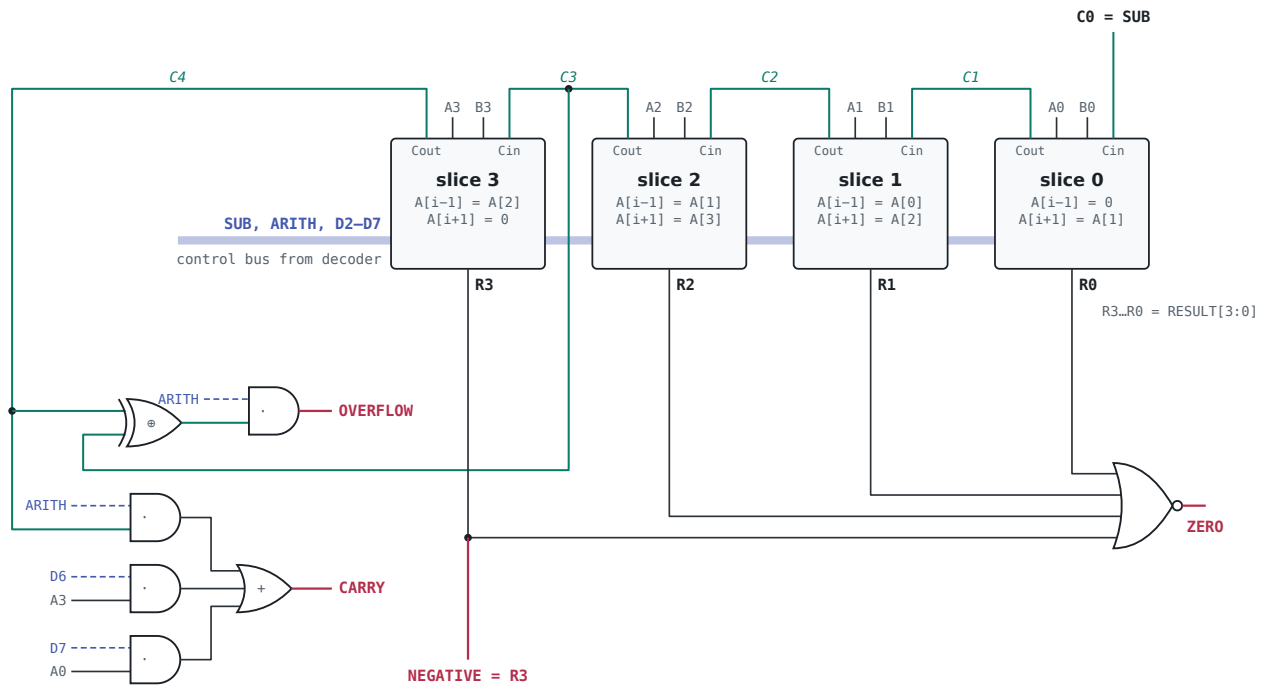
These diagrams show the single-cycle part of the ALU drawn with individual logic gates. They match the behaviour of the Verilog modules above (the earlier gate-only version was checked against the behavioural model on every input). In Figures 10 and 11, D2-D7 are the decoded select lines for opcodes 0010-0111 (AND, OR, XOR, NOT, SHL, SHR) and ARITH selects ADD/SUB.



**Figure 9.** control\_unit at gate level. Four inverters give each opcode bit and its complement; each AND gate matches one opcode pattern. arith and is\_muldiv ignore bit 0 because they cover two opcodes each.



**Figure 10.** One bit slice of the single-cycle datapath (repeated  $\times 4$ ). Top: adder/subtractor bit ( $B \text{ XOR } SUB$ , then a full adder). Left: logic gates. Bottom: shift wiring. Right: the result select as AND-OR. MUL/DIV results join at result\_mux.



**Figure 11.** Four slices, ripple carry and flag logic. The carry ripples from slice 0 to slice 3; C3 and C4 give the overflow flag. D6 = is\_shl and D7 = is\_shr.

## 11 Verification

The testbench `alu_4bit_tb.v` checks the design in three parts:

1. **Examples** — prints a readable table of each operation, including MUL, DIV and divide-by-zero.
2. **Single-cycle operations, exhaustive** — every opcode except MUL/DIV (14 opcodes, including the six unused ones) with all 256 combinations of a and b: 3,584 cases. Each is compared with a reference model written directly from the definitions (result, carry, overflow, zero, negative).
3. **MUL and DIV, exhaustive** — all 256 (a, b) pairs for each: 512 cases. It checks the product, the quotient and remainder against Verilog's `*`, `/` and `%`, the divide-by-zero behaviour, and that each operation takes exactly 5 clocks from start to done.

Output printed by the testbench (in Vivado it appears in the **Tcl Console** after **Run All**):

```
Op  code | A  B | result          | flags / notes
-----+-----+-----+-----+-----
ADD 0000 | 5  3 | 00001000 ( 8) | C=0 Z=0 V=1 N=1
ADD 0000 | 15 1 | 00000000 ( 0) | C=1 Z=1 V=0 N=0
SUB 0001 | 5  3 | 00000010 ( 2) | C=1 Z=0 V=0 N=0
SUB 0001 | 3  5 | 00001110 (14) | C=0 Z=0 V=0 N=1
AND 0010 | 12 10 | 00001000 ( 8) | C=0 Z=0 V=0 N=1
OR  0011 | 12 10 | 00001110 (14) | C=0 Z=0 V=0 N=1
XOR 0100 | 12 10 | 00000110 ( 6) | C=0 Z=0 V=0 N=0
NOT 0101 | 12  0 | 00000011 ( 3) | C=0 Z=0 V=0 N=0
SHL 0110 | 11  0 | 00000110 ( 6) | C=1 Z=0 V=0 N=0
SHR 0111 | 11  0 | 00000101 ( 5) | C=1 Z=0 V=0 N=0
MUL 1000 | 13 11 | 10001111 (143) | product = 143, 5 cycles
MUL 1000 | 15 15 | 11100001 (225) | product = 225, 5 cycles
DIV 1001 | 13  4 | 00010011      | quotient = 3, remainder = 1, dbz=0, 5 cycles
DIV 1001 | 15  3 | 00000101      | quotient = 5, remainder = 0, dbz=0, 5 cycles
DIV 1001 |  9  0 | 10011111      | quotient = 15, remainder = 9, dbz=1, 5 cycles

PASS: 3584 single-cycle checks + 512 MUL/DIV checks, 0 errors
```

## 12 How to run it in Vivado

1. **Create Project** → RTL Project → choose any part (or your board).
2. **Add Sources** → **Add or create design sources** → add the nine design files. Tick “Copy sources into project”.
3. **Add Sources** → **Add or create simulation sources** → add `alu_4bit_tb.v`.
4. Check the Sources window: `alu_4bit` is the design top and `alu_4bit_tb` is the simulation top (if not, right-click → **Set as Top**).
5. **Run Simulation** → **Run Behavioral Simulation**, then click **Run All**. Vivado stops at 1000 ns by default, and this test needs about 35  $\mu$ s. The PASS line appears in the Tcl Console.
6. **RTL Analysis** → **Open Elaborated Design** → **Schematic** shows the blocks of Figure 7. **Run Synthesis** and then **Report Utilization** show how many LUTs and flip-flops the ALU uses.

A separate easy guide, *“Verilog and Vivado — Easy Guide”*, explains every step with pictures.

## A Appendix: full source code

alu\_4bit.v

107 lines

```
`timescale 1ns / 1ps
// =====
// alu_4bit.v -- TOP LEVEL
// 4-bit ALU with 10 operations. MUL and DIV are done by
// shifting and re-using the ONE adder/subtractor of the ALU,
// instead of building a separate multiplier/divider array.
//
// opcode: 0000 ADD | 0001 SUB | 0010 AND | 0011 OR
//          0100 XOR | 0101 NOT | 0110 SHL | 0111 SHR
//          1000 MUL | 1001 DIV
//
// Single-cycle ops: result is ready immediately (combinational).
// MUL / DIV       : pulse 'start' for one clock, wait for 'done'
//                   (4 clock cycles of work).
// =====
module alu_4bit (
    input wire      clk,
    input wire      rst,
    input wire      start,
    input wire [3:0] a,
    input wire [3:0] b,
    input wire [3:0] opcode,
    output wire [7:0] result,
    output wire      carry,
    output wire      zero,
    output wire      overflow,
    output wire      negative,
    output wire      div_by_zero,
    output wire      busy,
    output wire      done
);
// ----- control -----
wire sub, arith, is_shl, is_shr, is_div, is_muldiv;

control_unit u_ctrl (
    .opcode(opcode), .sub(sub), .arith(arith),
    .is_shl(is_shl), .is_shr(is_shr),
    .is_div(is_div), .is_muldiv(is_muldiv)
);

// ----- shared adder / subtractor -----
// Normally fed from a, b. While MUL/DIV is busy it is fed
// by the muldiv unit instead (this is the hardware sharing).
wire [3:0] md_add_a, md_add_b;
wire      md_add_sub;
wire [3:0] add_in_a = busy ? md_add_a : a;
wire [3:0] add_in_b = busy ? md_add_b : b;
wire      add_sub   = busy ? md_add_sub : sub;
wire [3:0] sum;
wire      c3, c4;

adder_subtractor_4bit u_addsub (
    .a(add_in_a), .b(add_in_b), .sub(add_sub),
    .sum(sum), .c3(c3), .c4(c4)
);

// ----- logic and shift -----
wire [3:0] and_out, or_out, xor_out, not_out;
wire [3:0] shl, shr;
wire      shl_out_bit, shr_out_bit;

logic_unit_4bit u_logic (
    .a(a), .b(b),
```

```

        .and_out(and_out), .or_out(or_out),
        .xor_out(xor_out), .not_out(not_out)
    );

    shifter_4bit u_shift (
        .a(a), .shl(shl), .shr(shr),
        .shl_out_bit(shl_out_bit), .shr_out_bit(shr_out_bit)
    );

    // ----- multiply / divide -----
    wire [3:0] md_hi, md_lo;
    wire      md_dbz;

    muldiv_unit u_muldiv (
        .clk(clk), .rst(rst),
        .start(start & is_muldiv), .is_div(is_div),
        .a(a), .b(b),
        .add_a(md_add_a), .add_b(md_add_b), .add_sub(md_add_sub),
        .add_sum(sum), .add_c4(c4),
        .hi(md_hi), .lo(md_lo),
        .busy(busy), .done(done), .div_by_zero(md_dbz)
    );

    assign div_by_zero = is_div & md_dbz;

    // ----- result select -----
    result_mux u_mux (
        .opcode(opcode), .sum(sum),
        .and_out(and_out), .or_out(or_out),
        .xor_out(xor_out), .not_out(not_out),
        .shl(shl), .shr(shr),
        .md_hi(md_hi), .md_lo(md_lo),
        .result(result)
    );

    // ----- flags -----
    flag_unit u_flags (
        .result(result), .arith(arith),
        .is_shl(is_shl), .is_shr(is_shr), .is_muldiv(is_muldiv),
        .c3(c3), .c4(c4),
        .shl_out_bit(shl_out_bit), .shr_out_bit(shr_out_bit),
        .carry(carry), .zero(zero), .overflow(overflow), .negative(negative)
    );
endmodule

```

## control\_unit.v

36 lines

```

`timescale 1ns / 1ps
// =====
// control_unit.v
// Decodes the 4-bit opcode into the control signals used by
// the rest of the ALU.
//
// opcode | operation | type
// -----+-----+-----
// 0000 | ADD      | single cycle
// 0001 | SUB      | single cycle
// 0010 | AND      | single cycle
// 0011 | OR       | single cycle
// 0100 | XOR      | single cycle
// 0101 | NOT A    | single cycle
// 0110 | SHL      | single cycle
// 0111 | SHR      | single cycle
// 1000 | MUL      | multi-cycle (shift-and-add)
// 1001 | DIV      | multi-cycle (shift-and-subtract)
// 1010-1111 | unused -> result 0
// =====
module control_unit (
    input wire [3:0] opcode,
    output wire      sub,          // 1 for SUB          (0001)

```

```

output wire      arith,      // 1 for ADD or SUB      (000x)
output wire      is_shl,     // 1 for shift left      (0110)
output wire      is_shr,     // 1 for shift right     (0111)
output wire      is_div,     // 1 for divide          (1001)
output wire      is_muldiv   // 1 for MUL or DIV      (100x)
);
assign sub       = (opcode == 4'b0001);
assign arith     = (opcode[3:1] == 3'b000);
assign is_shl    = (opcode == 4'b0110);
assign is_shr    = (opcode == 4'b0111);
assign is_div    = (opcode == 4'b1001);
assign is_muldiv = (opcode[3:1] == 3'b100);
endmodule

```

## adder\_subtractor\_4bit.v

38 lines

```

`timescale 1ns / 1ps
// =====
// adder_subtractor_4bit.v
// 4-bit ripple-carry adder / subtractor.
//   sub = 0 : sum = a + b
//   sub = 1 : sum = a - b   (computed as a + ~b + 1, two's complement)
// Each bit of b is XORed with 'sub' (inverts b when subtracting),
// and 'sub' is also the carry-in to bit 0 (the "+1").
// Outputs c3 (carry into MSB) and c4 (carry out) for the flag unit.
// =====
module adder_subtractor_4bit (
    input wire [3:0] a,
    input wire [3:0] b,
    input wire      sub,
    output wire [3:0] sum,
    output wire      c3,    // carry into bit 3
    output wire      c4    // carry out of bit 3
);
    wire [3:0] b_x;        // b after optional inversion
    wire [4:0] c;          // internal carry chain

    assign c[0] = sub;

    // invert b when subtracting
    xor xb0 (b_x[0], b[0], sub);
    xor xb1 (b_x[1], b[1], sub);
    xor xb2 (b_x[2], b[2], sub);
    xor xb3 (b_x[3], b[3], sub);

    // ripple chain of four full adders
    full_adder fa0 (.a(a[0]), .b(b_x[0]), .cin(c[0]), .sum(sum[0]), .cout(c[1]));
    full_adder fa1 (.a(a[1]), .b(b_x[1]), .cin(c[1]), .sum(sum[1]), .cout(c[2]));
    full_adder fa2 (.a(a[2]), .b(b_x[2]), .cin(c[2]), .sum(sum[2]), .cout(c[3]));
    full_adder fa3 (.a(a[3]), .b(b_x[3]), .cin(c[3]), .sum(sum[3]), .cout(c[4]));

    assign c3 = c[3];
    assign c4 = c[4];
endmodule

```

## full\_adder.v

24 lines

```

`timescale 1ns / 1ps
// =====
// full_adder.v
// 1-bit full adder built from basic gates.
//   sum = a XOR b XOR cin
//   cout = (a AND b) OR ((a XOR b) AND cin)
// =====
module full_adder (
    input wire a,
    input wire b,
    input wire cin,

```

```

    output wire sum,
    output wire cout
);
    wire p;    // propagate = a xor b
    wire g;    // generate  = a and b
    wire pc;   // p and cin

    xor x1 (p,    a, b);
    xor x2 (sum,  p, cin);
    and a1 (g,    a, b);
    and a2 (pc,   p, cin);
    or  o1 (cout, g, pc);
endmodule

```

## logic\_unit\_4bit.v

19 lines

```

`timescale 1ns / 1ps
// =====
// logic_unit_4bit.v
// Bitwise logic operations, all computed in parallel.
// The result multiplexer picks the one the opcode asks for.
// =====
module logic_unit_4bit (
    input wire [3:0] a,
    input wire [3:0] b,
    output wire [3:0] and_out,    // a & b
    output wire [3:0] or_out,     // a | b
    output wire [3:0] xor_out,    // a ^ b
    output wire [3:0] not_out     // ~a
);
    assign and_out = a & b;
    assign or_out  = a | b;
    assign xor_out = a ^ b;
    assign not_out = ~a;
endmodule

```

## shifter\_4bit.v

22 lines

```

`timescale 1ns / 1ps
// =====
// shifter_4bit.v
// Logical shift by one position (pure wiring, no gates needed).
// Shift left : every bit moves one place left, 0 enters at bit 0,
//              a[3] falls off -> shl_out_bit  (a * 2)
// Shift right: every bit moves one place right, 0 enters at bit 3,
//              a[0] falls off -> shr_out_bit  (a / 2)
// Example a = 1011 : shl = 0110 (bit out 1), shr = 0101 (bit out 1)
// =====
module shifter_4bit (
    input wire [3:0] a,
    output wire [3:0] shl,
    output wire [3:0] shr,
    output wire      shl_out_bit,
    output wire      shr_out_bit
);
    assign shl      = {a[2:0], 1'b0};    // join lower 3 bits with a 0
    assign shr      = {1'b0, a[3:1]};    // join a 0 with upper 3 bits
    assign shl_out_bit = a[3];
    assign shr_out_bit = a[0];
endmodule

```

## muldiv\_unit.v

116 lines

```

`timescale 1ns / 1ps
// =====
// muldiv_unit.v
// Sequential multiplier and divider built from SHIFTING plus

```

```

// ONE 4-bit adder/subtractor. The adder is not inside this
// module: it is the ALU's own adder, borrowed while busy = 1.
// So MUL and DIV add only registers and a few muxes to the ALU.
//
// Registers
// acc : MUL -> upper half of product | DIV -> remainder
// q   : MUL -> multiplier, becomes the lower half of product
//       DIV -> dividend, becomes the quotient
// m   : MUL -> multiplicand | DIV -> divisor
//
// MULTIPLY (shift-and-add), 4 steps, one per clock:
// if q[0] == 1 : acc = acc + m (add multiplicand)
// shift {carry, acc, q} right by 1
// after 4 steps: product = {acc, q}
//
// DIVIDE (restoring, shift-and-subtract), 4 steps:
// shift {acc, q} left by 1 (bring down next dividend bit)
// try acc - m
// if it fits (no borrow): acc = acc - m, q[0] = 1
// else : keep acc, q[0] = 0
// after 4 steps: quotient = q, remainder = acc
//
// Protocol: put a, b on the inputs, pulse start for one clock.
// busy goes high for 4 clocks, then done goes high and the
// result is held until the next start.
// Divide by zero: div_by_zero = 1, quotient = 1111, remainder = a.
// =====
module muldiv_unit (
    input wire clk,
    input wire rst, // active-high asynchronous reset
    input wire start, // pulse for one clock to begin
    input wire is_div, // 0 = multiply, 1 = divide
    input wire [3:0] a,
    input wire [3:0] b,

    // connection to the shared adder/subtractor
    output wire [3:0] add_a,
    output wire [3:0] add_b,
    output wire add_sub,
    input wire [3:0] add_sum,
    input wire add_c4, // carry out (for SUB: 1 = no borrow)

    output wire [3:0] hi, // MUL: product[7:4] DIV: remainder
    output wire [3:0] lo, // MUL: product[3:0] DIV: quotient
    output wire busy,
    output reg done,
    output reg div_by_zero
);
    reg [3:0] acc, q, m;
    reg [1:0] count; // counts the 4 steps (0..3)
    reg running;
    reg mode_div; // operation latched at start

    // divide: {acc, q} shifted left by one
    wire [3:0] r_shift = {acc[2:0], q[3]};
    wire r_top = acc[3]; // bit shifted out of acc
    wire fits = r_top | add_c4; // remainder >= divisor ?

    // drive the shared adder
    assign add_a = mode_div ? r_shift : acc;
    assign add_b = m;
    assign add_sub = mode_div; // subtract when dividing

    assign busy = running;
    assign hi = acc;
    assign lo = q;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            acc <= 4'd0; q <= 4'd0; m <= 4'd0;
            count <= 2'd0; running <= 1'b0; mode_div <= 1'b0;

```

```

        done <= 1'b0;  div_by_zero <= 1'b0;
    end
    else if (start && !running) begin
        // load
        running      <= 1'b1;
        done         <= 1'b0;
        mode_div     <= is_div;
        count        <= 2'd0;
        acc          <= 4'd0;
        q            <= is_div ? a : b;    // dividend / multiplier
        m            <= is_div ? b : a;    // divisor / multiplicand
        div_by_zero <= is_div && (b == 4'd0);
    end
    else if (running) begin
        if (!mode_div) begin
            // ---- one multiply step: add (if q[0]) then shift right
            if (q[0])
                {acc, q} <= {add_c4, add_sum, q[3:1]};
            else
                {acc, q} <= {1'b0, acc, q[3:1]};
        end
        else begin
            // ---- one divide step: shift left, subtract if it fits
            if (fits) begin
                acc <= add_sum;           // r_shift - m
                q <= {q[2:0], 1'b1};
            end
            else begin
                acc <= r_shift;           // restore (keep)
                q <= {q[2:0], 1'b0};
            end
        end
    end

    count <= count + 2'd1;
    if (count == 2'd3) begin
        running <= 1'b0;
        done <= 1'b1;
    end
end
end
endmodule

```

## result\_mux.v

31 lines

```

`timescale 1ns / 1ps
// =====
// result_mux.v
// Chooses which unit's output becomes the ALU result.
// The result is 8 bits wide so it can hold a full product.
// single-cycle ops : result = {0000, 4-bit answer}
// MUL               : result = {product high, product low}
// DIV               : result = {remainder, quotient}
// =====
module result_mux (
    input wire [3:0] opcode,
    input wire [3:0] sum,
    input wire [3:0] and_out, or_out, xor_out, not_out,
    input wire [3:0] shl, shr,
    input wire [3:0] md_hi, md_lo,
    output reg [7:0] result
);
    always @(*) begin
        case (opcode)
            4'b0000, 4'b0001: result = {4'b0000, sum};
            4'b0010:          result = {4'b0000, and_out};
            4'b0011:          result = {4'b0000, or_out};
            4'b0100:          result = {4'b0000, xor_out};
            4'b0101:          result = {4'b0000, not_out};
            4'b0110:          result = {4'b0000, shl};
            4'b0111:          result = {4'b0000, shr};

```

```

        4'b1000, 4'b1001: result = {md_hi, md_lo};
    default:               result = 8'b0000_0000;
endcase
end
endmodule

```

## flag\_unit.v

33 lines

```

`timescale 1ns / 1ps
// =====
// flag_unit.v
// Status flags.
//  carry    : ADD -> carry out, SUB -> 1 = no borrow,
//             SHL/SHR -> bit shifted out, others -> 0
//  overflow  : signed overflow on ADD/SUB (C3 xor C4), others 0
//  zero      : whole 8-bit result is 0
//  negative  : result[3] for 4-bit ops (sign bit);
//             0 for MUL/DIV (unsigned)
// =====
module flag_unit (
    input wire [7:0] result,
    input wire      arith,
    input wire      is_shl,
    input wire      is_shr,
    input wire      is_muldiv,
    input wire      c3,
    input wire      c4,
    input wire      shl_out_bit,
    input wire      shr_out_bit,
    output wire      carry,
    output wire      zero,
    output wire      overflow,
    output wire      negative
);
    assign carry    = (arith & c4)
                     | (is_shl & shl_out_bit)
                     | (is_shr & shr_out_bit);
    assign overflow = arith & (c3 ^ c4);
    assign zero     = ~(result);           // NOR of all 8 bits
    assign negative = ~is_muldiv & result[3];
endmodule

```

## alu\_4bit\_tb.v

161 lines

```

// =====
// alu_4bit_tb.v
// Testbench for alu_4bit.
// Part 1: prints example operations, including MUL and DIV.
// Part 2: checks every single-cycle op on all 256 (a,b) pairs.
// Part 3: checks MUL and DIV on all 256 (a,b) pairs, including
//         the latency and divide-by-zero.
//
//
// HOW TO RUN IN VIVADO
// 1. Add this file as a SIMULATION source.
// 2. Flow Navigator -> Run Simulation -> Run Behavioral Simulation.
// 3. The simulation stops after 1000 ns by default. Click
//    "Run All" (or type run all in the Tcl Console) so the
//    whole test runs (about 35 us). Results print in the Tcl Console.
// =====
`timescale 1ns/1ps

module alu_4bit_tb;
    reg      clk = 0, rst = 1, start = 0;
    reg [3:0] a = 0, b = 0, opcode = 0;
    wire [7:0] result;
    wire      carry, zero, overflow, negative, div_by_zero, busy, done;

    alu_4bit dut (

```

```

        .clk(clk), .rst(rst), .start(start),
        .a(a), .b(b), .opcode(opcode),
        .result(result), .carry(carry), .zero(zero),
        .overflow(overflow), .negative(negative),
        .div_by_zero(div_by_zero), .busy(busy), .done(done)
    );

    always #5 clk = ~clk;           // 100 MHz clock (10 ns period)

    // ----- reference model for single-cycle ops -----
    reg [3:0] exp_r;
    reg      exp_c, exp_v;
    reg [4:0] t;
    task model;
    begin
        exp_c = 0; exp_v = 0; exp_r = 0;
        case (opcode)
            4'b0000: begin t = {1'b0,a} + {1'b0,b};          exp_r = t[3:0]; exp_c = t[4];
                        exp_v = (a[3] == b[3]) && (exp_r[3] != a[3]); end
            4'b0001: begin t = {1'b0,a} + {1'b0,~b} + 5'd1; exp_r = t[3:0]; exp_c = t[4];
                        exp_v = (a[3] != b[3]) && (exp_r[3] != a[3]); end
            4'b0010: exp_r = a & b;
            4'b0011: exp_r = a | b;
            4'b0100: exp_r = a ^ b;
            4'b0101: exp_r = ~a;
            4'b0110: begin exp_r = {a[2:0], 1'b0}; exp_c = a[3]; end
            4'b0111: begin exp_r = {1'b0, a[3:1]}; exp_c = a[0]; end
            default: exp_r = 0;
        endcase
    end
endtask

    // ----- run one MUL or DIV, count clock cycles -----
    integer cycles;
    task run_md(input [3:0] op, input [3:0] x, input [3:0] y);
    begin
        @(negedge clk);
        opcode = op; a = x; b = y; start = 1;
        @(negedge clk);
        start = 0; cycles = 0;
        while (!done) begin @(negedge clk); cycles = cycles + 1; end
    end
endtask

    task show_comb(input [3:0] op, input [3:0] x, input [3:0] y, input [8*4-1:0] name);
    begin
        opcode = op; a = x; b = y; #1;
        $display(" %s %b | %2d %2d | %b (%3d) | C=%b Z=%b V=%b N=%b",
            name, op, x, y, result, result, carry, zero, overflow, negative);
    end
endtask

    task show_md(input [3:0] op, input [3:0] x, input [3:0] y, input [8*4-1:0] name);
    begin
        run_md(op, x, y);
        if (op == 4'b1000)
            $display(" %s %b | %2d %2d | %b (%3d) | product = %0d, %0d cycles",
                name, op, x, y, result, result, result, cycles + 1);
        else
            $display(" %s %b | %2d %2d | %b          | quotient = %0d, remainder = %0d, dbz=%b, %0d
cycles",
                name, op, x, y, result, result[3:0], result[7:4], div_by_zero, cycles + 1);
    end
endtask

    integer i, errors, op_i;
    reg [7:0] ab;           // loop value split into a (ab[7:4]) and b (ab[3:0])

    initial begin
        #12 rst = 0;

```

```

// ----- Part 1: examples -----
$display("\n Op   code |  A   B |  result          | flags / notes");
$display("-----+-----+-----+-----");
show_comb(4'b0000, 5, 3, "ADD ");
show_comb(4'b0000, 15, 1, "ADD ");
show_comb(4'b0001, 5, 3, "SUB ");
show_comb(4'b0001, 3, 5, "SUB ");
show_comb(4'b0010, 12, 10, "AND ");
show_comb(4'b0011, 12, 10, "OR  ");
show_comb(4'b0100, 12, 10, "XOR ");
show_comb(4'b0101, 12, 0, "NOT ");
show_comb(4'b0110, 11, 0, "SHL ");
show_comb(4'b0111, 11, 0, "SHR ");
show_md  (4'b1000, 13, 11, "MUL ");
show_md  (4'b1000, 15, 15, "MUL ");
show_md  (4'b1001, 13, 4, "DIV ");
show_md  (4'b1001, 15, 3, "DIV ");
show_md  (4'b1001, 9, 0, "DIV ");

errors = 0;

// ----- Part 2: every single-cycle op, all inputs -----
for (op_i = 0; op_i < 16; op_i = op_i + 1) begin
    if (op_i != 8 && op_i != 9) begin
        for (i = 0; i < 256; i = i + 1) begin
            opcode = op_i; {a, b} = i; #1;
            model;
            if (result != {4'b0, exp_r} || carry != exp_c || overflow != exp_v ||
                zero != (exp_r == 0) || negative != exp_r[3]) begin
                errors = errors + 1;
                if (errors <= 10)
                    $display("FAIL op=%b a=%0d b=%0d got %b expected %b", opcode, a, b, result,
exp_r);
            end
        end
    end
end
end

// ----- Part 3: MUL and DIV, all inputs -----
for (i = 0; i < 256; i = i + 1) begin
    ab = i;
    run_md(4'b1000, ab[7:4], ab[3:0]);
    if (result != ab[7:4] * ab[3:0] || cycles + 1 != 5 || zero != (result == 0)) begin
        errors = errors + 1;
        if (errors <= 10) $display("FAIL MUL %0d*%0d got %0d", ab[7:4], ab[3:0], result);
    end
    run_md(4'b1001, ab[7:4], ab[3:0]);
    if (ab[3:0] == 0) begin
        if (div_by_zero != 1 || result[3:0] != 4'b1111 || result[7:4] != ab[7:4]) begin
            errors = errors + 1;
            if (errors <= 10) $display("FAIL DIV %0d/0", ab[7:4]);
        end
    end
    else if (result[3:0] != ab[7:4] / ab[3:0] || result[7:4] != ab[7:4] % ab[3:0] ||
        div_by_zero != 0) begin
        errors = errors + 1;
        if (errors <= 10) $display("FAIL DIV %0d/%0d got q=%0d r=%0d",
            ab[7:4], ab[3:0], result[3:0], result[7:4]);
    end
end
end

if (errors == 0)
    $display("\nPASS: 3584 single-cycle checks + 512 MUL/DIV checks, 0 errors\n");
else
    $display("\nFAILED: %0d errors\n", errors);
$finish;

end
endmodule

```